PolicyBurst: Constraint-First Cloud Bursting from a Private Kubernetes Cluster under Cost, Latency and Data-Residency Constraints

Ramanuj Agarwal (23BCE1861)

Guide: Indra Priyadarshini S.

Cloud Computing Project P11

Code and data: github.com/AviatorRoyal/cloud-burst

Abstract

Organisations that run a private cloud usually size it for typical demand and borrow public capacity during peaks. The controllers that make this cloud-bursting decision in practice react to a single utilisation or queue threshold. They have no notion of the rules that govern the decision inside an institution: a monthly spending ceiling, a latency objective that distant regions cannot meet, and a requirement that some categories of data stay inside India. This paper presents PolicyBurst, a bursting controller for Kubernetes that takes these three constraints as inputs to the decision instead of checking them afterwards. For every workload it first builds the set of permissible destinations from the data-classification label and measured round-trip times, then acquires capacity only inside that set and only if the projected spend for the full lease stays under the ceiling. We implemented PolicyBurst against a five-node k3s cluster with a tainted public node pool and evaluated it in two ways: 9 measured runs on the cluster and 227 runs of a trace-driven simulator that executes the same decision code. PolicyBurst recorded no residency violations and no ceiling breaches in any run, while a threshold-only baseline served on average 7163 restricted requests per testbed run from foreign nodes and spent 2.1 times the ceiling under repeated bursts. When the ceiling binds, PolicyBurst delivers +80% more SLO attainment per rupee than the baseline. Without a binding ceiling the gain depends on burst size: +41% at 1.5 times private capacity but −13% at 2.5 times, where a static latency filter discards slow but useful capacity. Enforcing residency costs almost nothing until the restricted share of traffic exceeds what the in-country public node can absorb, after which the attainment penalty rises to 32 percentage points at a 70% share.

Keywords: cloud bursting, hybrid cloud, Kubernetes, data residency, DPDP Act 2023, cost-aware scheduling, autoscaling

1. Introduction

A university or a hospital that runs its own private cloud faces an awkward sizing problem. Hardware bought for the busiest week of the year sits idle for most of the other fifty-one; hardware bought for an ordinary week leaves students staring at time-outs on the day results are published. Cloud bursting is the usual compromise. The private cluster is sized for typical load, and when demand rises above it the overflow is sent to rented public capacity, which is released again when the peak passes.

Someone, or something, has to decide when to burst, how much to rent and which work to send out. In most deployments that decision is made by an autoscaler with a single trigger: add a public node when CPU utilisation crosses a threshold or when pods cannot be scheduled. The trigger is easy to reason about and it keeps pods running. What it cannot do is respect the constraints that actually govern the decision in an institution. Three of them come up repeatedly.

- **A spending ceiling.** Public capacity is paid from a budget approved in advance. Finance offices care less about the average bill than about whether a month can exceed its allocation, and a threshold controller has no reason to stop renting when the money runs out.
- **A latency objective.** A public region that is cheap but far away may add more delay than the service level allows. Sending work there spends money without improving what users experience.
- **Data residency.** Some data must not leave the country. Section 16 of India's Digital Personal Data Protection Act 2023 lets the government restrict transfers of personal data to notified countries, and sectoral rules such as the Reserve Bank of India's 2018 directive on payment data already require storage within India. Many institutions also adopt stricter internal rules for student or patient records.

A controller that ignores these constraints can make decisions that are operationally correct, because the pods do run, but administratively or legally unacceptable. The usual remedy is to filter decisions after they are made, for example by refusing to schedule a pod on a forbidden node. Post-hoc filtering is weak in two ways. The capacity has already been bought by the time the filter rejects it, so the budget is spent anyway. And a filter that only says no cannot redirect the work to a destination that would have been acceptable.

This paper takes the opposite approach and treats the constraints as inputs. Our controller, PolicyBurst, first computes for every workload the set of destinations it may use given its data class and the measured round-trip time to each public node. Only then does it decide whether to burst, and it buys capacity only within that set and only if the projected cost of the whole lease fits under the ceiling. When the set is empty the request is rejected with an explicit reason, which makes the cost of compliance measurable instead of hidden.

The work answers four objectives set out in the problem statement: to design a controller that admits cost, latency and residency constraints as explicit inputs (O1); to implement it against a private Kubernetes cluster with a public node pool (O2); to evaluate service-level attainment per

unit of expenditure under varying burst intensity (O3); and to quantify the capacity and attainment cost of enforcing data residency (O4). The contributions are:

1. A constraint-first decision procedure for cloud bursting in which residency and latency define a feasible destination set per workload and a cost accumulator gates every acquisition on projected rather than current spend (Section 5).
2. An open-source implementation, PolicyBurst, consisting of a Kubernetes custom resource for the policy and a Python controller of about 750 lines with 28 unit tests. A threshold baseline runs through the same code path with the constraints switched off, so comparisons isolate the decision logic (Section 6).
3. An evaluation on a real five-node k3s cluster and in a trace-driven simulator that executes the unchanged decision code, covering 227 simulated runs across burst sizes, burst durations, an ablation and a residency sweep (Sections 7 and 8).
4. A quantified answer to what residency compliance costs: almost nothing while restricted traffic fits on in-country public capacity, and a steep penalty once it does not (Section 8.8).

The rest of the paper is organised as follows. Section 2 gives background on bursting, the Kubernetes mechanisms we rely on and the residency setting. Section 3 reviews related work. Section 4 formalises the problem and states the hypotheses. Sections 5 and 6 describe the design and implementation, Section 7 the methodology and Section 8 the results. Section 9 discusses what the results mean, Section 10 lists threats to validity and Section 11 concludes.

2. Background

2.1 Cloud bursting

Cloud bursting extends a private or on-premises deployment with public capacity during demand peaks. It differs from moving a workload to the cloud permanently: the private resources remain the default home of the workload, and public capacity is temporary and paid by the hour or second. Three decisions have to be made repeatedly. The first is when to burst, which requires a signal that the private side is saturated and a judgement about whether the saturation will last long enough to justify the delay and cost of new capacity. The second is how much capacity to acquire. The third is which work to send out, because not every service or request is equally suitable for an external location.

Most deployed systems answer the first question with a threshold and the second with a fixed increment, and they ignore the third. This is reasonable when all public capacity is equivalent and all work is equally portable. It stops being reasonable when public destinations differ in price and distance, and when some work carries obligations about where it may run.

2.2 Kubernetes mechanisms

PolicyBurst is built entirely from standard Kubernetes primitives, so it can run on any conformant cluster. Four are central.

- **Taints and tolerations.** A taint on a node repels pods that do not tolerate it. Public nodes carry the taint p11.burst/public=true:NoSchedule, so system pods and unrelated workloads never land on rented capacity by accident.
- **Cordoning.** Setting spec.unschedulable on a node stops new pods from being placed there without touching running pods. PolicyBurst keeps public nodes cordoned until it decides to use them, which models acquiring and releasing capacity.
- **Node affinity.** A required node-affinity rule restricts a Deployment's pods to nodes whose labels match. Each node carries p11.burst/destination and p11.burst/region labels, and the controller pins every managed Deployment to its permissible destinations.
- **Horizontal Pod Autoscaler (HPA).** The HPA scales the number of replicas from CPU utilisation. With the default algorithm the desired replica count is the current count multiplied by the ratio of observed to target utilisation, scale-up is limited to doubling or four extra pods per fifteen seconds, and scale-down waits for a five-minute stabilisation window. When the HPA asks for more pods than the private nodes can hold, the extra pods stay Pending, which is the clearest signal that the private side is full.

The division of labour is therefore simple. The HPA decides how many replicas a service needs. The Kubernetes scheduler decides where each replica goes within the nodes that are schedulable and permitted. PolicyBurst decides which public nodes are schedulable at all and which of them each service is permitted to use.

2.3 Data residency in the Indian setting

The Digital Personal Data Protection Act 2023 does not impose a general localisation requirement. Its Section 16 allows the central government to restrict transfers of personal data to countries it notifies, and it preserves any other law that provides a higher degree of protection or restriction. Sector regulators already use that room: the Reserve Bank of India's 2018 directive requires payment system data to be stored only in India. For an institution the practical consequence is a mixture of hard legal limits for some data, contractual or policy limits for other data, and no limit for public information. PolicyBurst models this with a small set of data classes, each mapped to the regions where it may be processed. We do not claim that pod placement alone amounts to legal compliance; Section 10 returns to this point.

3. Related Work

We reviewed fifteen papers published from 2021 onwards that touch on hybrid-cloud scheduling, cost-aware bursting, multi-cluster Kubernetes and constrained placement. Table 1 summarises each by method, evaluation setting, metric and the limitation most relevant to this work.

Table 1. Literature summary (15 papers, 2021 onwards).

Paper	Method	Setting	Metric	Limitation
Stoica & Shenker, HotOS 2021	Vision of an intercloud broker (sky computing)	Position paper	Qualitative	No controller or evaluation

Paper	Method	Setting	Metric	Limitation
Wiesner et al., Middleware 2021	Temporal shifting of delay-tolerant jobs to low-carbon hours	Simulation, 2020 grid data	CO2 saved	Single objective; no budget, latency or residency
Ju, Singh & Toor, UCC 2021	Forecast-driven proactive pod autoscaler	Kubernetes edge cluster	Utilisation, response time	One cluster; no external capacity
Toka et al., IEEE TNSM 2021	ML forecasting for scaling edge clusters	Kubernetes edge clusters	Lost requests, resources	No cost or placement policy
Phuc, Phan & Kim, IEEE Access 2022	Traffic-aware HPA across edge nodes	Kubernetes edge testbed	Throughput, latency	Ignores cost and jurisdiction
Xiao et al., FGCS 2022	Privacy-preserving workflow scheduling across DCs	Simulation	Makespan, cost	Offline; no live bursting
Iorio et al., IEEE TCC 2023 (Liqo)	Peering Kubernetes clusters, virtual-node offloading	Real multi-cluster K8s	Overhead, latency	Mechanism without policy
Yang et al., NSDI 2023 (SkyPilot)	Broker picks cheapest cloud and region per job	AWS, GCP, Azure	Cost, completion time	Batch jobs, public only
Yang et al., IEEE TCC 2023	PSO cost minimisation under deadline	Simulated cloud	Cost, deadline	Single cloud; no residency
TanGo, IEEE INFOCOM 2023	Submodular tenant task placement	Simulation, multi-region	Electricity cost	Sovereignty not modelled
Tekawade & Banerjee, COMSNETS 2023	Reliability-aware multi-cloud scheduling	Simulation	Cost, reliability	No private/public split
Luo et al., USENIX ATC 2024 (Starburst)	Waiting budget decides which jobs burst	Traces + 32-GPU cluster	Cloud cost, JCT	No hard ceiling or residency
Wu et al., NSDI 2024	Spot instances with on-demand fallback	SkyPilot, public cloud	Cost, deadlines met	No jurisdiction or private cluster
Abdi, Ashjaei & Mubeen, FGCS 2024	Keeps sensitive workflow tasks private	Simulation	Monetary cost	Offline, simulated only
Sukprasert et al., EuroSys 2024	Temporal and spatial carbon-aware shifting	Carbon-intensity traces	Carbon reduction	Analysis, no controller

Three strands stand out. The first is cost-aware use of public capacity. Starburst, by Luo and colleagues, is the closest system to ours: a scheduler for a private GPU cluster that decides which jobs to send to the cloud by giving large jobs a longer waiting budget than small ones. It optimises a trade-off between cloud cost and job completion time, but it has no hard spending ceiling and treats every cloud location as equivalent. SkyPilot and the spot-instance work of Wu and colleagues choose the cheapest place to run a batch job, again without any notion of where data may go, and without a private cluster that remains the default home of the work.

The second strand is placement under privacy or security constraints. Abdi, Ashjaei and Mubeen keep security-sensitive workflow tasks on private resources while minimising monetary cost under a deadline, and Xiao and colleagues restrict data movement between geo-distributed data centres for privacy. Both are offline planners evaluated in simulation: they decide where each task of a known workflow should run before execution starts. A bursting controller has no such luxury, because demand arrives unannounced and capacity has to be bought and released as it happens.

The third strand provides mechanisms. Liqo, described by Iorio and colleagues, peers Kubernetes clusters and lets pods be offloaded transparently through a virtual node. It shows that the plumbing for multi-cluster placement exists, but it deliberately leaves the placement policy to the operator.

The edge autoscalers of Ju, Toka and Phuc improve scaling inside one cluster and do not consider external capacity at all.

Across the table no work enforces a hard spending ceiling, a per-destination latency check and a data-residency rule together, in a live controller over a private and a public Kubernetes pool. That gap is what PolicyBurst addresses. Residency in particular appears in the literature mostly as motivation or as a manually configured region list, not as a first-class input whose cost is measured.

4. Problem Formulation

4.1 System model

The system consists of a set of destinations D. One or more are private; the rest are public. Each destination d has a region r(d), a shadow price p(d) in rupees per hour, and a measured round-trip time t(d) from the private ingress. Private destinations cost nothing at the margin and add negligible delay. Workloads are Kubernetes Deployments; each workload w carries a data class c(w) such as restricted, internal or public. The policy maps each class c to a set of permitted regions R(c). Requests to a workload are served by its pods, and a request meets the service-level objective (SLO) if it succeeds within L milliseconds. The policy also fixes a spending ceiling B for the billing period, a reserve ρ kept below the ceiling, a minimum lease length λ for public capacity, and an assumed in-pod service time s.

4.2 Constraints

A destination is permissible for workload w if it satisfies both the residency and the latency constraint:

$$F(w) = \{\, d \in D : r(d) \in R(c(w)) \;\wedge\; k \cdot t(d) + s \le L \,\} \qquad (1)$$

Here k is the number of round trips a forwarded request pays; through kube-proxy a request typically pays one round trip for the TCP handshake and one for the request itself, so we use k = 2. A workload whose data class is missing or unknown is treated as restricted, so a labelling mistake fails closed. Private destinations always satisfy the latency constraint.

Let S(τ) be the spend accumulated up to time τ and let A be the set of public destinations currently in use. A new destination d may be acquired only if the projected spend over one more lease stays below the ceiling minus the reserve:

$$S(\tau) + \left(\Sigma_{a \in A}\, p(a) + p(d)\right) \cdot \min(\lambda,\, T - \tau) \;\le\; B - \rho \qquad (2)$$

where T is the end of the billing period. The projection matters because a node acquired just under the ceiling still bills for its whole lease. A controller that checks only the current spend S(τ) can overshoot by up to one lease per node. In addition, all public capacity is released whenever the next control interval Δ would cross the limit:

$$S(\tau) + \Sigma_{a \in A}\, p(a) \cdot \Delta \;>\; B - \rho \;\Rightarrow\; \text{release all of } A \qquad (3)$$

4.3 Objective and metrics

Within these constraints the controller should maximise SLO attainment, defined as the share of all requests that succeed within L, with failed and rejected requests counted as misses. Because spending more can always buy more attainment, the main efficiency metric is attainment per unit of expenditure, reported as attainment percentage points per ₹100. We also record expenditure, public node-hours, the 95th-percentile latency of successful requests, the number of restricted requests served from a non-permitted region (residency violations), whether the ceiling was breached, and the number of requests that failed while the controller reported that no permissible destination existed. For O4 the attainment penalty of residency is the difference in attainment between the controller with and without the residency constraint, everything else equal.

4.4 Hypotheses

The problem statement asks for falsifiable claims of the form 'method X reduces metric M by at least N% compared with baseline B under condition C'. We fixed three before running the experiments:

- **H1 (efficiency).** PolicyBurst improves SLO attainment per rupee by at least 25% over threshold-only bursting under bursts of 1.5 to 4 times private capacity with a fixed ceiling.
- **H2 (compliance).** PolicyBurst records zero ceiling breaches and zero residency violations in every run, while the threshold baseline breaches the ceiling whenever cumulative burst load exceeds the budget.
- **H3 (cost of compliance).** With 30% of traffic tagged restricted, enforcing residency lowers SLO attainment by at most 10 percentage points compared with the same controller without the residency constraint.

5. Design of PolicyBurst

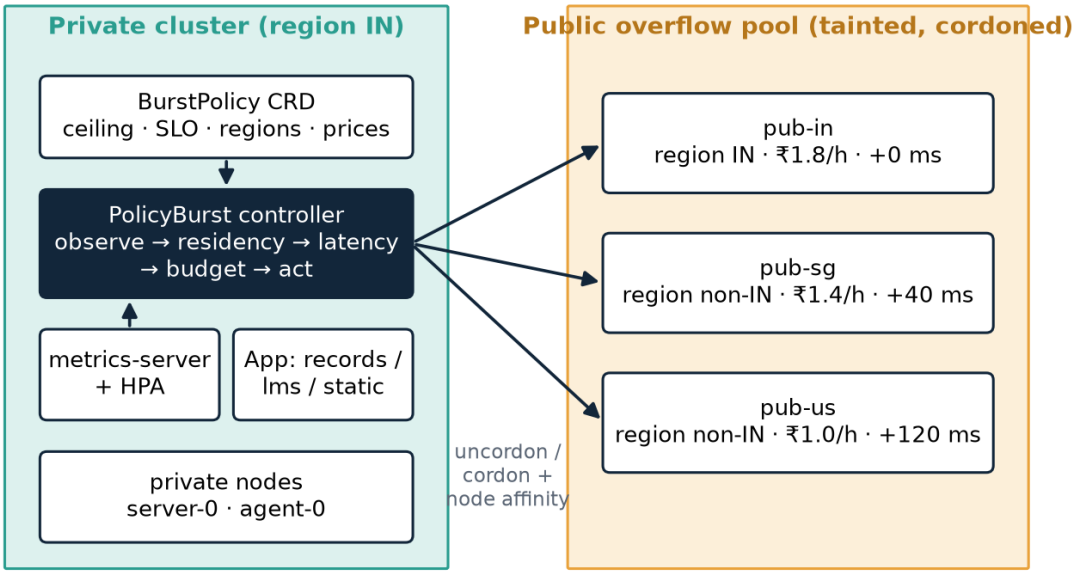


Fig. 1. Architecture. The controller reads the BurstPolicy and cluster state, and acts by cordoning or uncordoning public nodes and by pinning each Deployment to its permissible destinations.

5.1 Overview

Figure 1 shows the architecture. The policy lives in the cluster as a custom resource of kind BurstPolicy, so an administrator changes the ceiling, the price list or the residency map with kubectl apply and the controller picks the change up on its next cycle. The controller runs a reconcile loop every fifteen seconds. Each cycle has five steps: observe, residency filter, latency filter, budget gate and act. The first four steps are pure functions of the observation and the policy, which made them straightforward to test and allowed the same code to drive the simulator described in Section 7.

5.2 Observe

The controller lists the managed Deployments (those labelled p11.burst/managed=true) and counts their pods that are Pending with reason Unschedulable. It reads node CPU usage from metrics-server and averages it over the private nodes. It reads which public destinations are currently schedulable, and once a minute it measures the round-trip time to each public node by timing TCP connections to the kubelet port, taking the median of three attempts. For safe scale-in it also sums the CPU requested by managed pods on each public destination and the unrequested CPU left on the private nodes.

5.3 Feasible set per workload

For every workload the controller evaluates Equation 1. The residency filter keeps destinations whose region is permitted for the workload's data class. The latency filter keeps destinations whose doubled round-trip time plus service time fits the SLO; a destination that has not been probed or did not answer is excluded. The resulting set is written to the Deployment as a required node-affinity rule over the destination label. Because changing a pod template rolls the Deployment's pods, the controller records the current set in an annotation and patches only when the set changes, which in practice happens when the policy changes or a destination's latency moves across the threshold.

If the residency or latency filter removes every public destination, the controller records the reason. A workload in that state can still use private capacity, but any of its requests that cannot be served are attributed to the absence of a permissible destination. This is what makes the cost of compliance visible in the metrics.

5.4 When and how much to burst

Pressure is declared when any managed workload has pending pods or private CPU utilisation exceeds 75%. A burst is triggered only after pressure has lasted thirty seconds, which filters out the transient pending pods that appear for a few seconds during every HPA scale-up. Capacity is added one destination per reconcile cycle, and a further destination is considered only after the previous one has had thirty seconds to absorb pending pods. For each workload with pending pods the controller sorts its permissible public destinations by price and picks the cheapest one that is not yet active and that passes the budget gate of Equation 2. The cheapest-first order is what a

cost-conscious operator would do by hand; the difference from the baseline lies entirely in which destinations are permissible.

5.5 Cost accumulator

The cost accumulator meters each active public destination per second at its shadow price and persists its state to a file, so a controller restart does not forget money already spent. It exposes the projected-spend test of Equation 2 for acquisitions and the halt test of Equation 3 for running capacity. When the halt test fires, every public destination is drained and cordoned, its pods return to Pending, and they are rescheduled onto private capacity or wait. New acquisitions refused by the gate are logged with reason 'budget'.

5.6 Safe scale-in

Capacity is released when no workload has pending pods and private utilisation has stayed below 40% for a configurable period. The most expensive active destination goes first. An early version released the destination as soon as the idle condition held. Simulation exposed a problem with this: the HPA keeps replicas for five minutes after load falls, so draining a public node while the HPA still held its replicas sent those pods back to Pending, which immediately looked like pressure and triggered a new burst. The controller now releases a destination only if the CPU requested by its managed pods fits into the unrequested CPU of the private nodes; otherwise it waits and logs the reason.

5.7 Rollout and placement guards

PolicyBurst narrows a workload's permissible destinations by patching the Deployment's pod template. Kubernetes applies such a change as a rolling update, and until the rollout completes the old ReplicaSet keeps pods with the old, wider affinity. If the HPA scales the Deployment during the rollout, some new replicas can be created from the old template. When the controller opens a foreign node at that moment, those replicas are allowed to land on it. The testbed exposed exactly this: in two of three early PolicyBurst runs, 62 and 126 restricted requests were served from a non-Indian node. The simulator could not show the problem because it does not model rollouts.

Two guards close the gap. The rollout guard refuses to acquire public capacity while residency is enforced and any managed Deployment is mid-rollout, or its current template allows a different destination set from the one just computed. The placement guard runs every cycle and evicts any managed pod found on a destination its workload may not use, so a misplaced pod is corrected within one interval even if it slips through. All PolicyBurst results reported in Section 8 were measured with both guards; the earlier runs are kept separately in the repository.

5.8 Baselines in the same code path

The threshold-only baseline (B1) is the same reconcile loop with all three constraints disabled: every destination is permissible for every workload, and the budget gate always passes. The no-bursting baseline (B0) never uncordons a public node. Ablations disable one constraint at a time. Running baselines through the same code means that actuation, timing, measurement and logging

are identical across configurations, so differences in the results come from the decision logic alone.

The core decision logic, simplified, is:

```
for each workload w:
    allowed[w] = residency_ok(w) ∩ latency_ok(w)        # constraints first
if pending pods or private CPU > 75% for 30 s:
    for w in workloads with pending pods:
        for d in sorted(allowed[w] ∩ public, by price):
            if d inactive and projected_spend(d) ≤ B − ρ:
                activate(d); break
        else: reject(w, reason)                          # residency | latency |
budget
if spend after next interval > B − ρ: release all public
if idle 2 min and pods on priciest node fit on private: release it
```

6. Implementation

PolicyBurst is written in Python on the official Kubernetes client library and is about 750 lines in the controller package. The package is split into pure modules for the policy model, feasibility filters, cost accumulator and decision procedure, and I/O modules that read cluster state and apply actions. The pure modules are covered by 28 unit tests that run without a cluster, including tests for fail-closed handling of unknown data classes, the projected-spend gate, the budget halt, the one-destination-per-cycle rule and the scale-in safety check. The repository also contains the BurstPolicy custom resource definition, two policies (a calendar-month policy at real-scale prices and a time-compressed experiment policy), the sample application, the Locust load generator, the experiment runner, the simulator and the analysis scripts. All of it is version-controlled at github.com/AviatorRoyal/cloud-burst.

Actuation uses three API operations. Activating a destination patches its nodes' spec.unschedulable to false; deactivating sets it to true and evicts managed pods through the Eviction API so that PodDisruptionBudgets are respected. Restricting a workload patches its Deployment with a required node-affinity term over p11.burst/destination and a toleration for the public taint. The controller can run as a Deployment inside the cluster with a narrowly scoped ClusterRole, or as a process on the server, which is how the experiment runner uses it so that each run starts from a fresh cost meter.

The sample application has three services that stand in for an institution's workloads: records (restricted, for example student records linked to identity numbers), lms (internal course content) and static (public pages). Each is a small Python HTTP server that performs a fixed computation of about 15 ms of CPU time per request (the iteration count is calibrated against process CPU time when the pod starts), declares a CPU request of 250m and a limit of 500m, and is scaled by an HPA targeting 60% CPU with one to thirty replicas. The script ships in a ConfigMap on a stock python:3.12-slim image, so the testbed needs no image registry.

The load generator is Locust with a custom load shape. Each simulated user sends about two requests per second, closed-loop: the next request waits for the previous response or a five-second timeout. A run consists of a two-minute warm-up at base load, one or more bursts of a chosen size

and duration, and a three-minute cool-down. The fraction of requests sent to the restricted service is a parameter. Every request is logged with its timestamp, service, latency, outcome and the node that served it, which is how residency violations are counted from the client side rather than trusted from the controller.

7. Experimental Methodology

7.1 Testbed

The testbed is a real k3s cluster (v1.35.5) of five nodes created with k3d on a single host (16 vCPU, WSL2 Ubuntu 26.04). Each node is a container running a full k3s server or agent, with its CPU capped at two cores and its allocatable CPU set to match. Two nodes form the private pool (labelled destination=private, region=IN). Three form the public pool: pub-in in region IN with no added delay, pub-sg in region non-IN with 40 ms of added delay, and pub-us in region non-IN with 120 ms of added delay. The delays are applied with Linux netem on each node's host-side network interface, so every packet into the node pays them, including the controller's latency probes and requests forwarded by kube-proxy. The measured round-trip times were below 0.2 ms for private and pub-in, about 40 to 50 ms for pub-sg and 120 to 150 ms for pub-us. Public nodes carry the public taint and start cordoned.

The repository also includes scripts that build the same topology from real virtual machines joined over a Tailscale overlay, with a free-tier VM in an Indian region as pub-in. The results in this paper come from the single-host testbed; Section 10 discusses what that changes.

7.2 Simulator

Running every combination of burst size, duration, configuration and repetition on the testbed would take several days, because each run lasts about nine minutes of wall-clock time. We therefore built a trace-driven simulator that executes the unchanged PolicyBurst decision code and cost accumulator against a one-second model of the cluster. The model covers the components whose timing drives the results: the HPA with its scale-up limit and five-minute scale-down window, the scheduler's least-allocated placement over permitted and schedulable nodes, pod start-up of three to five seconds plus a one-off image pull on a fresh public node, CPU limits per pod and per node, closed-loop users, a fluid request queue per service and node with a five-second client timeout, M/M/1-style service time inflation, log-normal latency noise with a small heavy tail, and noisy round-trip probes. The simulated private pool has seven cores and the public nodes four each, matching the cloud deployment the scripts target rather than the smaller testbed.

Two modelling errors were found and fixed while building the simulator, and both are worth recording because they change results substantially. An early version capped throughput per pod but not per node, which let a saturated node serve more requests than its cores allowed. Another used open-loop arrivals, which made overload look like a total collapse; real Locust users slow down when responses slow down, so attainment under overload settles around a third rather than falling to zero. The testbed comparison in Section 8.3 is the check that the corrected model is close enough.

7.3 Workloads and configurations

Capacity was calibrated by ramping load with bursting disabled and taking the largest user count at which attainment stayed at or above 95%. This gave 60 users on the testbed and 220 users in the simulator (Figure 2). Burst sizes are expressed as multiples of this capacity, so a 2× burst on the testbed means 120 users. Base load is about a fifth of capacity, and 30% of requests go to the restricted service unless stated otherwise.

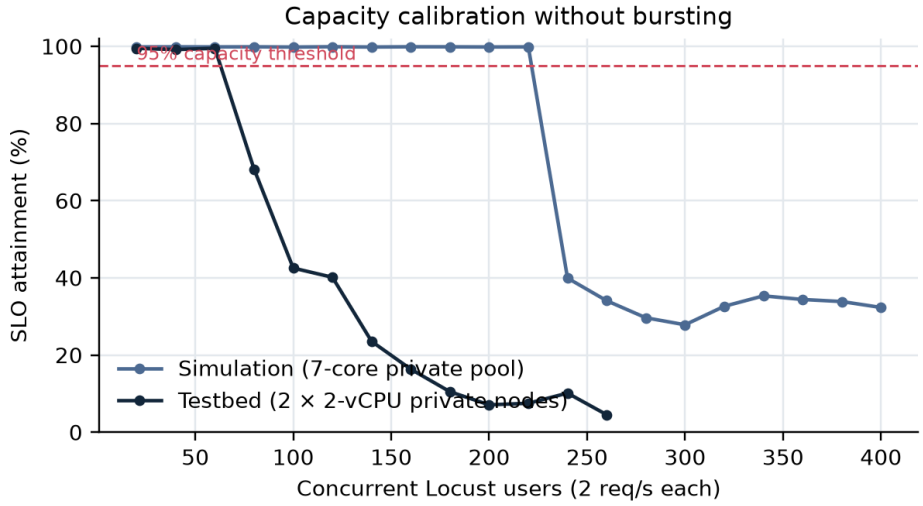


Fig. 2. Capacity calibration with bursting disabled. Capacity is the largest load with at least 95% SLO attainment.

The configurations are B0 (no bursting), B1 (threshold-only), PB (PolicyBurst) and three ablations: PB without the residency filter (PB-noRes), without the latency filter (PB-noLat) and without the cost accumulator (PB-noCost). All runs use the time-compressed experiment policy, in which one billing period lasts an hour, the ceiling is ₹120 with a ₹10 reserve, the minimum lease is six minutes, and shadow prices are ₹180, ₹140 and ₹100 per hour for pub-in, pub-sg and pub-us. These are the real-scale prices of ₹1.8, ₹1.4 and ₹1.0 per hour multiplied by one hundred, which keeps their ratios while letting the ceiling bind within a run. The SLO is 300 ms.

Table 2. Experiment sets.

Set	Configurations	Bursts	Restricted	Reps	Runs
Testbed	B0, B1, PB	2× for 180 s	30%	3	9
Simulation: main matrix	B0, B1, PB, PB-noRes	1.5×, 2×, 2.5×, 3×, 4× × 60, 180, 300 s	30%	3	180
Simulation: ablation	B1, PB, PB-noCost, PB-noLat, PB-noRes	five 2× bursts of 180 s	30%	3	15
Simulation: residency sweep	PB, PB-noRes	2× for 180 s	0–70%	3	30
Simulation: spend trace	B1, PB	five 2× bursts of 180 s	30%	1	2

7.4 Statistics

Each point is the mean over three repetitions with a 95% confidence interval from the t-distribution. Within a repetition all configurations use the same random seed for arrivals and noise, so comparisons between configurations are paired. Three repetitions are few, and the intervals are

wide where the outcome depends on start-up timing, which is mainly at 2× and 2.5× bursts. We report the intervals rather than hide them.

8. Results

8.1 Testbed: baselines and PolicyBurst

Table 3. Testbed results, 2× burst for 180 s, mean of three runs (± 95% CI for attainment).

Configuration	Attainment (%)	p95 (ms)	Failed req.	Node-hours	Spend (₹)	Violations
B0 no bursting	66.7 ± 14.5	1782	335	0.000	0.0	0
B1 threshold-only	92.8 ± 3.5	362	0	0.328	44.4	7163
PolicyBurst	88.3 ± 14.0	658	82	0.297	41.0	0

Table 3 gives the measured results on the cluster. Without bursting, attainment over the whole run was 66.7%: the private pool served the warm-up and cool-down well but fell behind as soon as the burst began. Threshold-only bursting raised attainment by 26.1 points to 92.8% and spent ₹44.4 per run. Its first purchase is always the cheapest node, pub-us. On this testbed pub-us has a measured round trip of 119 ms, so a forwarded request pays about 239 ms of network delay and still fits inside the 300 ms objective; pub-us is therefore useful capacity here, and PolicyBurst's own latency filter admits it as well ($2 \times 119 + 20 = 259$ ms). It also served on average 7163 restricted requests per run from nodes outside India. PolicyBurst reached 88.3% for ₹41.0, 4.6 points below the threshold baseline, with 0 residency violations.

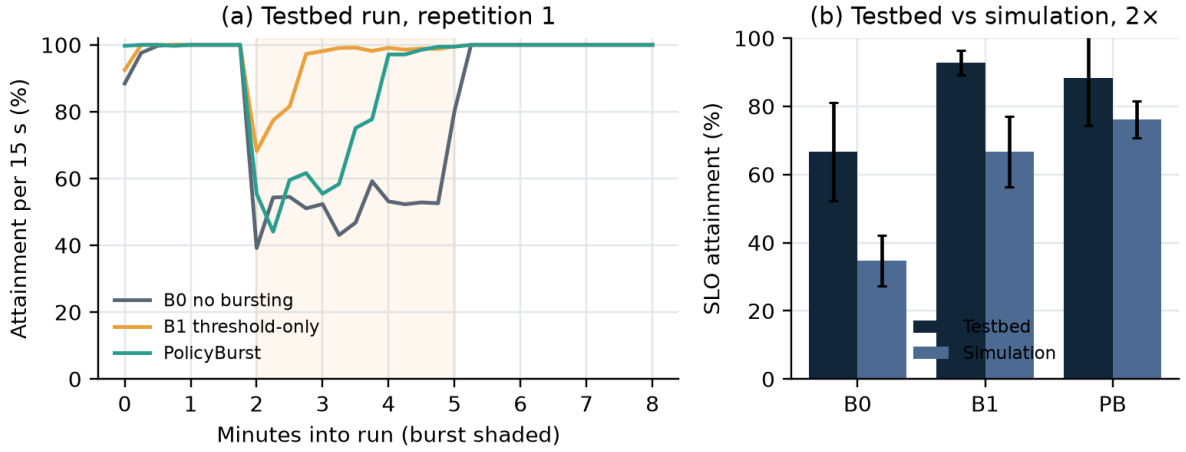


Fig. 3. (a) SLO attainment per 15-second window in one testbed run of each configuration; the burst is shaded. (b) Mean attainment on the testbed and in the simulator for the same 2× burst.

Figure 3(a) shows how the runs unfold. All three configurations behave identically during warm-up. When the burst starts, the private pool saturates within seconds, pods go Pending as the HPA asks for more replicas than fit, and attainment drops. B0 stays down for the whole burst. The bursting configurations recover once the controller has seen thirty seconds of pressure and a public node has accepted the pending pods, which takes about a minute in total including pod start-up.

8.2 Calibration

The calibration curves in Figure 2 have the same shape on both platforms. Attainment stays near 100% up to capacity and then falls steeply once the private nodes run out of CPU: on the testbed it was 68.2% at 80 users, and in the simulator 40.0% at 240 users. The two platforms differ in absolute capacity (60 against 220 users) because the testbed's private pool has four cores and the modelled one seven, which is why burst sizes are always expressed relative to each platform's own capacity.

8.3 Does the simulator agree with the testbed?

Figure 3(b) compares the two platforms at the one point both cover. The simulator gives 34.7%, 66.7% and 76.2% for B0, B1 and PolicyBurst; the testbed gives 66.7%, 92.8% and 88.3%. The largest gap between the platforms is 32.0 percentage points. The absolute values therefore differ noticeably, which we attribute to the smaller testbed nodes, real pod start-up times and contention on the shared host; the simulated numbers in the following sections should be read as indicating trends rather than as predictions of a particular cluster. The ranking of the three configurations is not the same on both platforms, which limits how far the simulated comparisons can be trusted. On both platforms the threshold baseline served restricted requests from foreign nodes and PolicyBurst did not. The main reason the two controllers rank differently on the two platforms is the emulated round trip to pub-us. The simulator models a cloud path with a 28 ms base round trip plus the 120 ms delay, which puts pub-us outside the SLO and lets PolicyBurst gain by avoiding it. The single-host testbed has no base round trip, so pub-us measured 119 ms and stayed inside the SLO; there the latency filter has nothing to remove, and PolicyBurst gives up some attainment because restricted pods may not use the two foreign nodes.

Building this comparison also exposed a flaw in an early version of the test application. Its request handler spun on the wall clock for 15 ms instead of performing a fixed amount of computation. When a pod reached its CPU limit and was throttled, the wall clock kept running, so each request silently did less work and the service never saturated the way a CPU-bound service should. The handler now calibrates a fixed iteration count against process CPU time at start-up, and all testbed results reported here were measured after the fix; the earlier runs were discarded.

8.4 Attainment per rupee across burst sizes (O3)

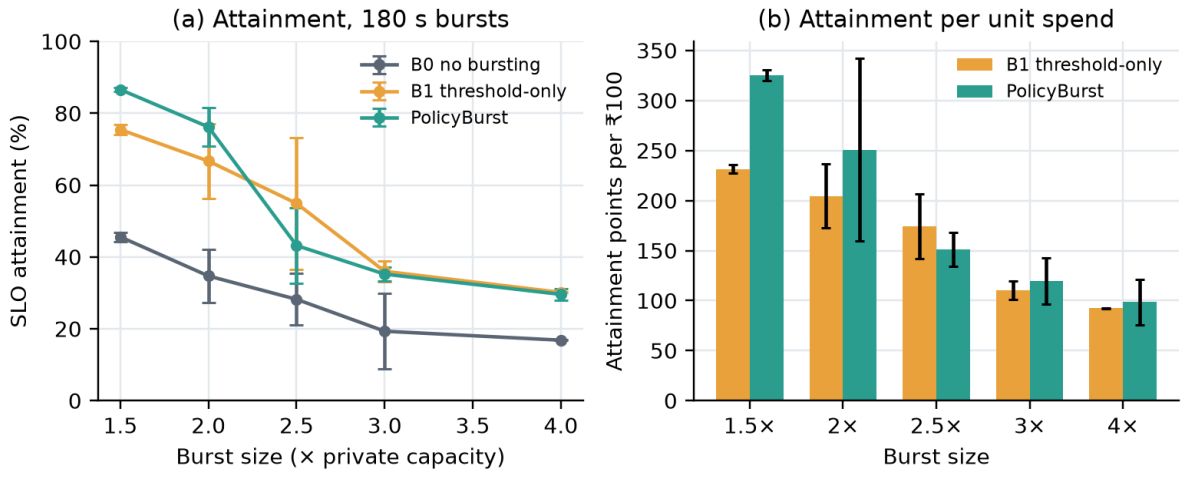


Fig. 4. Simulation, 180 s bursts. (a) SLO attainment against burst size. (b) Attainment points per ₹100 for the two bursting configurations. Error bars are 95% confidence intervals over three repetitions.

Table 4. Simulation, 180 s bursts: attainment (%) and attainment per ₹100.

Burst	B0 att.	B1 att.	PB att.	B1 per ₹100	PB per ₹100	PB vs B1
1.5×	45.6	75.5	86.6	231.6	325.4	+41%
2×	34.7	66.7	76.2	204.7	250.8	+23%
2.5×	28.2	54.9	43.2	174.2	150.9	−13%
3×	19.3	36.0	35.2	110.5	119.5	+8%
4×	16.8	30.1	29.6	92.5	98.8	+7%

Figure 4 and Table 4 show the main result for O3. At moderate bursts PolicyBurst is clearly better: at 1.5× it reaches 86.6% against 75.5% for the baseline, and it does so while spending less, giving +41% attainment per rupee. At 2× the advantage is +23%, though the confidence interval is wide. The reason is the latency filter. The threshold baseline picks the cheapest public node, pub-us, whose round trip of about 150 ms means that every request forwarded there pays roughly 300 ms of network delay and misses the SLO. PolicyBurst skips pub-us and uses pub-sg and pub-in, so the capacity it pays for actually serves requests within the objective.

At 2.5× the picture reverses: PolicyBurst gets 43.2% against 54.9% and −13% attainment per rupee. At this load the two acceptable public nodes are not enough, and the requests queued on saturated nodes wait longer than the extra 300 ms that pub-us would have added. A request served slowly from a distant node still counts as a miss, but it relieves the queue for everyone else. The static latency filter cannot see this trade-off because it compares round-trip time with the SLO without considering the local queueing delay. At 3× and 4× everything is overloaded and the configurations converge, with PolicyBurst +8% and +7% per rupee.

8.5 Burst duration

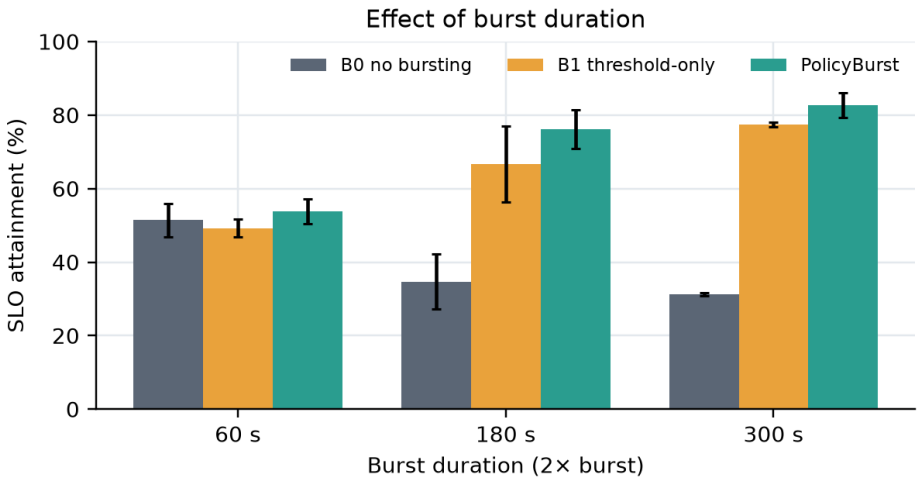


Fig. 5. Simulation, 2× bursts of different duration.

Figure 5 varies the duration of a 2× burst. Short bursts are the hardest case for any reactive controller. A sixty-second burst is half over by the time the controller has seen thirty seconds of pressure and a new node has started its pods, so bursting recovers only part of the lost attainment.

As bursts get longer the fixed reaction delay matters less and the gap between bursting and not bursting widens. PolicyBurst's attainment per rupee relative to the baseline was +26% for 60 s bursts and +6% for 300 s bursts.

8.6 Enforcing the ceiling

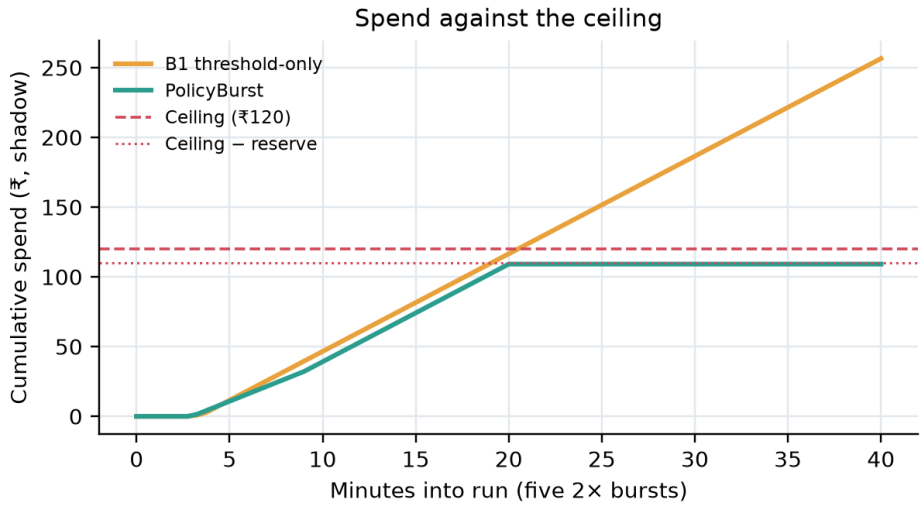


Fig. 6. Simulation, five 2× bursts in one compressed billing period. Cumulative spend of the threshold baseline and PolicyBurst against the ₹120 ceiling.

Single bursts never reach the ₹120 ceiling, so to test the cost accumulator we ran five 2× bursts of 180 s in one compressed billing period. Figure 6 shows the result. The two controllers spend at the same rate for the first twenty minutes. The threshold baseline keeps going and ends at ₹255, 2.1 times the ceiling. PolicyBurst stops acquiring when the projected spend of a further lease would cross ₹110 (the ceiling minus the reserve) and finishes at ₹109. Later bursts are absorbed by private capacity alone or are rejected with reason 'budget'. The ceiling was breached in every baseline run and in no PolicyBurst run.

8.7 Ablation: what causes the gain?

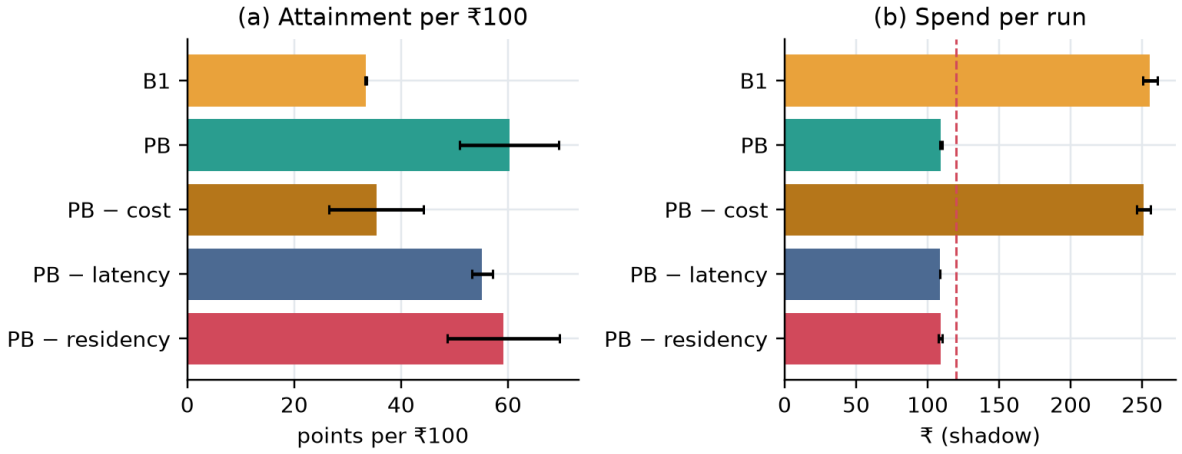


Fig. 7. Simulation ablation under five 2× bursts with the ceiling binding. (a) Attainment per ₹100. (b) Spend per run; the dashed line is the ceiling.

Table 5. Ablation under repeated bursts (mean of three runs).

Configuration	Attainment (%)	Spend (₹)	Per ₹100	Violations	Ceiling breached
B1 threshold-only	85.4	255.4	33.4	108012	yes
PolicyBurst	65.9	109.3	60.3	0	no
PB without cost accumulator	88.8	251.1	35.3	0	yes
PB without latency filter	60.1	108.8	55.2	0	no
PB without residency	64.4	108.9	59.2	57186	no

Removing one component at a time (Figure 7, Table 5) shows where the efficiency comes from. Without the cost accumulator, PolicyBurst behaves like the baseline on spend: it reaches 88.8% attainment but spends ₹251 and breaches the ceiling in every run, and its attainment per rupee falls from 60.3 to 35.3. The accumulator is therefore the main source of the +80% per-rupee advantage over the baseline when the ceiling binds. Without the latency filter, PolicyBurst buys the cheap distant node and its attainment drops from 65.9% to 60.1% for the same budget. Without the residency filter efficiency is similar to full PolicyBurst, but 57186 restricted requests per run are served from foreign nodes.

The ablation also shows a cost of PolicyBurst that should not be hidden. Its absolute attainment under repeated bursts is lower than the baseline's, because once the budget is gone later bursts get no public capacity. Whether that is the right outcome is a policy decision: the ceiling exists precisely because the institution has decided that some degradation is preferable to overspending. PolicyBurst makes the trade explicit; the baseline simply spends.

8.8 The cost of residency (O4)

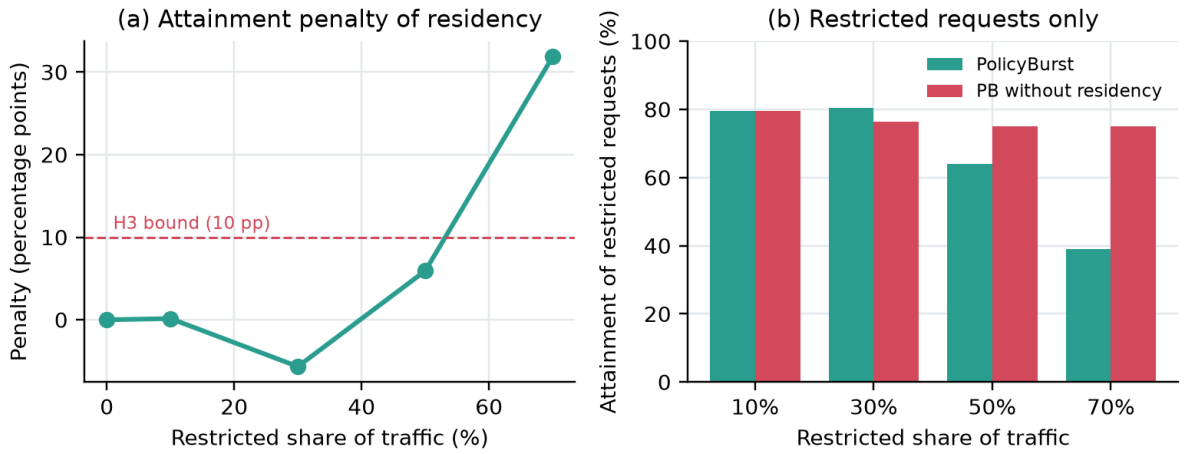


Fig. 8. Simulation, 2× bursts of 180 s with an increasing share of restricted traffic. (a) Attainment penalty from enforcing residency. (b) Attainment of restricted requests only.

Table 6. Residency sweep: attainment (%) with and without the residency constraint.

Restricted share	PB	PB without residency	Penalty (pp)	PB restricted	No-res restricted
0%	76.0	76.0	0.0	n/a	n/a
10%	76.2	76.4	0.1	79.5	79.5
30%	76.2	70.5	-5.7	80.6	76.4
50%	69.0	75.0	5.9	64.1	75.1

Restricted share	PB	PB without residency	Penalty (pp)	PB restricted	No-res restricted
70%	44.1	76.0	31.9	39.0	75.1

For O4 we varied the share of requests that go to the restricted service from 0% to 70% and compared PolicyBurst with and without the residency constraint (Figure 8, Table 6). Up to a 30% share the penalty is within noise: 0.1 points at 10% and -5.7 at 30%. Restricted pods can still use private capacity and pub-in, and that is enough for the restricted load. At 50% the penalty becomes 5.9 points, and at 70% it reaches 31.9 points. At that share the restricted service needs more capacity than the private pool and the single in-country public node can provide, while the cheaper foreign node sits available but forbidden. The attainment of restricted requests alone falls to 39.0% with residency enforced, against 75.1% without it.

The number of requests rejected because no permissible destination existed was zero in the sweep, because pub-in was always a permissible destination for restricted work. The cost of compliance showed up as degraded service on the in-country capacity rather than as outright rejections. Rejections did appear in the repeated-burst runs, where the budget made every public destination impermissible.

8.9 Hypotheses

H1 is partly supported. On the testbed at a 2× burst PolicyBurst delivered 228.8 attainment points per ₹100 against 210.1 for the threshold baseline (+9%). In simulation the required 25% gain holds at 1.5× bursts (+41%) and when the ceiling binds (+80%). It is close at 2× (+23%, with a wide interval) and fails at 2.5× (−13%), 3× (+8%) and 4× (+7%).

H2 is supported. PolicyBurst recorded zero residency violations and zero ceiling breaches in all 227 simulated runs and all 3 PolicyBurst runs on the testbed. The threshold baseline served on average 7163 restricted requests per testbed run from foreign nodes, and it breached the ceiling in 100% of the repeated-burst runs.

H3 is supported at the stated 30% share, with a penalty of -5.7 points against the 10-point bound. The sweep shows that this conclusion does not extend to higher shares: at 50% the penalty is 5.9 points and at 70% it is 31.9.

9. Discussion

The results support the main claim of the design: when constraints are inputs to the bursting decision rather than filters on its output, the controller can honour them without giving up much service. Compliance with the ceiling and with residency was perfect across every run, and in the common case of moderate bursts the latency filter actually improved service per rupee by refusing capacity that could not meet the objective. These benefits come from information the threshold controller does not use, which is the point of making the constraints explicit.

The failure of H1 at 2.5× is the most useful finding for future work. The latency filter answers the question 'can this destination meet the SLO?' in isolation. Under heavy overload the right question

is 'will a request served from this destination be faster than the same request waiting here?'. A distant node that adds 300 ms is a poor choice when local latency is 50 ms and a good one when local queues add two seconds. A queue-aware filter that compares the extra network delay with the observed queueing delay on private capacity would keep PolicyBurst's behaviour at moderate load and recover the baseline's advantage at heavy load. The data needed for this is already collected by the controller.

The residency sweep gives institutions a practical rule. The cost of keeping restricted data in the country depends on whether in-country capacity, private or public, can absorb the restricted share of peak demand. Below that point the cost is close to zero. Above it, the cost is steep and grows with the share. An institution can therefore estimate its compliance cost by measuring the restricted share of peak traffic, and it can reduce the cost most effectively by adding in-country public capacity rather than by any change to the controller.

The cost results show the value of projecting spend rather than checking it. A controller that tests only current spend would have acquired its last node just under the ceiling and then overshot by the rest of that node's lease. PolicyBurst stops early enough that it never crosses the line, at the price of leaving a small part of the budget unused. The reserve is the knob that sets this margin.

Two practical observations came out of building the system. Running the baseline through the same reconcile loop with the constraints switched off removed a whole class of confounding differences in timing and actuation, and we recommend this pattern for anyone comparing controllers. And the two evaluation layers caught different bugs. The simulator found the scale-in thrash described in Section 5.6, which unit tests had missed; the testbed found the rollout race described in Section 5.7, which the simulator could not represent. Neither layer alone would have produced a correct controller.

10. Threats to Validity

- **Simulation model.** Most results come from a simulator that approximates request service with fluid queues and an M/M/1-style service time. The testbed check covers one burst size. The direction of effects agreed, but absolute values differ between the platforms.
- **Single-host testbed.** All testbed nodes share one physical host, and the foreign regions are emulated with netem delay. Real wide-area links add jitter, loss and asymmetric routes that the emulation does not reproduce, and contention on the shared host can couple nodes in ways separate machines would not.
- **Modelled cost.** Free-tier usage costs nothing, so spend is shadow-priced, and the experiment policy compresses a month into an hour with prices scaled one hundred times. Real bills include egress, storage and rounding to billing increments.
- **Synthetic load.** Bursts are square waves on top of constant base load. Real demand ramps up, overlaps between services and follows daily cycles, which favours controllers that react early.

- **Residency is not legal compliance.** PolicyBurst controls where pods run. Logs, backups, monitoring data and the Kubernetes control plane itself may still cross borders, and a real compliance argument needs all of them.
- **Scale and repetitions.** Five nodes, three services and three repetitions per point are small. Confidence intervals are wide at the burst sizes where outcomes depend on start-up timing.

11. Conclusion and Future Work

This paper presented PolicyBurst, a cloud-bursting controller for Kubernetes that takes a spending ceiling, a latency objective and data-residency rules as inputs to the decision. It builds a feasible destination set per workload before deciding to burst, buys capacity only within that set, and gates every purchase on projected spend over the full lease. On a five-node k3s testbed and in 227 simulated runs it never violated residency and never exceeded its ceiling, while a threshold controller did both. When the budget binds it delivers +80% more attainment per rupee than the threshold controller; at moderate bursts it is also more efficient, by +41% at 1.5× capacity, but its static latency filter costs it at heavier bursts. The cost of residency compliance is small until restricted demand exceeds in-country capacity and large beyond that point.

Three extensions follow directly. The first is the queue-aware latency filter discussed in Section 9, which should close the gap at heavy bursts. The second is to replace the emulated foreign regions with real public nodes, starting with a free-tier VM in an Indian region joined over the overlay network the repository already supports, and to repeat the testbed comparison across all burst sizes. The third is to extend residency enforcement beyond pod placement to the storage, logging and monitoring paths that a full compliance argument would need.

References

[1] I. Stoica and S. Shenker, "From cloud computing to sky computing," in Proc. Workshop on Hot Topics in Operating Systems (HotOS), 2021.

[2] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska and L. Thamsen, "Let's wait awhile: How temporal workload shifting can reduce carbon emissions in the cloud," in Proc. 22nd Int. Middleware Conf., 2021, pp. 260–272.

[3] L. Ju, P. Singh and S. Toor, "Proactive autoscaling for edge computing systems with Kubernetes," in Proc. IEEE/ACM 14th Int. Conf. Utility and Cloud Computing Companion (UCC), 2021.

[4] L. Toka, G. Dobreff, B. Fodor and B. Sonkoly, "Machine learning-based scaling management for Kubernetes edge clusters," IEEE Trans. Network and Service Management, vol. 18, no. 1, 2021.

[5] L. H. Phuc, L.-A. Phan and T. Kim, "Traffic-aware horizontal pod autoscaler in Kubernetes-based edge computing infrastructure," IEEE Access, vol. 10, 2022.

[6] Y. Xiao, A. C. Zhou, X. Yang and B. He, "Privacy-preserving workflow scheduling in geo-distributed data centers," Future Generation Computer Systems, vol. 130, pp. 46–58, 2022.

[7] M. Iorio, F. Risso, A. Palesandro, L. Camiciotti and A. Manzalini, "Computing without borders: The way towards liquid computing," IEEE Trans. Cloud Computing, vol. 11, no. 3, pp. 2820–2838, 2023.

[8] Z. Yang et al., "SkyPilot: An intercloud broker for sky computing," in Proc. 20th USENIX Symp. Networked Systems Design and Implementation (NSDI), 2023.

[9] Yang, Xia, Ye, Gao and Zhan, "Workflow scheduling in clouds: cost minimisation under deadline constraints with HPCP-PSO," IEEE Trans. Cloud Computing, 2023.

[10] "TanGo: A cost optimization framework for tenant task placement in geo-distributed clouds," in Proc. IEEE INFOCOM, 2023.

[11] Tekawade and Banerjee, "A cost-effective reliability-aware scheduler for task graphs in multi-cloud systems," in Proc. Int. Conf. Communication Systems and Networks (COMSNETS), 2023.

[12] M. Luo et al., "Starburst: A cost-aware scheduler for hybrid cloud," in Proc. USENIX Annual Technical Conf. (ATC), 2024.

[13] Z. Wu et al., "Can't be late: Optimizing spot instance savings under deadlines," in Proc. 21st USENIX Symp. Networked Systems Design and Implementation (NSDI), 2024, pp. 185–203.

[14] S. Abdi, M. Ashjaei and S. Mubeen, "Deadline-constrained security-aware workflow scheduling in hybrid cloud architecture," Future Generation Computer Systems, 2024.

[15] T. Sukprasert, A. Souza, N. Bashir, D. Irwin and P. Shenoy, "On the limitations of carbon-aware temporal and spatial workload shifting in the cloud," in Proc. 19th European Conf. Computer Systems (EuroSys), 2024.

[16] Government of India, The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023).

[17] Reserve Bank of India, "Storage of payment system data," circular RBI/2017-18/153, April 2018.

[18] The Kubernetes Authors, "Horizontal Pod Autoscaling" and "Taints and tolerations," Kubernetes documentation, kubernetes.io.

[19] K3s Project, "K3s: Lightweight Kubernetes," docs.k3s.io; k3d project, k3d.io.

[20] Locust Authors, "Locust: An open source load testing tool," locust.io.

[21] S. Hemminger, "Network emulation with NetEm," in Proc. Linux Conf. Australia (linux.conf.au), 2005.

[22] L. Kleinrock, Queueing Systems, Volume 1: Theory. New York: Wiley, 1975.